

MAVEN

BUILD AUTOMATION

Lifecycle · Dependency Management · Plugins · Docker Integration

Why Build Tools Exist

The problems that triggered automation

Manual Compilation Pain

Developers had to remember exact javac commands, classpaths, and dependencies for every build — error-prone and time-consuming.

Dependency Hell

Managing JAR files manually led to version conflicts, missing libraries, and the infamous 'works on my machine' syndrome.

Inconsistent Builds

Without a standard process, builds differed between developers, CI servers, and production — causing unpredictable failures.

Repetitive Tasks

Testing, packaging, and deploying were manual steps that had to be repeated identically every release cycle.

Maven solves all of the above through a standardized, declarative build system.

Project Object Model (POM)

The heart of every Maven project — pom.xml

GroupId: Unique org identifier e.g. com.example

ArtifactId: Project/module name e.g. my-app

Version: Release version e.g. 1.0.0-SNAPSHOT

Packaging: Output format: jar, war, pom

Dependencies: All library declarations live here

Build: Plugin config, source dirs, output

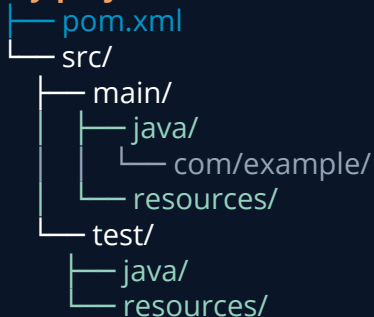
Profiles: Environment-specific configurations

```
<project>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <dependencies>
    <dependency>
      ...
    </dependency>
  </dependencies>
</project>
```

Standard Maven Directory Structure

Convention over configuration

my-project/



target/ ← generated output

src/main/java

Production Java source code. Maven compiles everything here.

src/main/resources

Config files, properties, XMLs bundled into the artifact.

src/test/java

Unit and integration test classes (not bundled in output).

src/test/resources

Test-only config files and fixtures.

target/

Auto-generated build output — classes, JARs, WAR files.

pom.xml

Project descriptor — the single source of truth for the build.

Build Lifecycle Phases

Maven's three built-in lifecycles: default, clean, site

1 validate

Checks project is correct & all required info is available

2 compile

Compiles source code of the project into .class files

3 test

Runs unit tests using a testing framework (e.g., JUnit)

4 package

Bundles compiled code into distributable format (JAR/WAR)

5 verify

Runs integration checks to ensure quality criteria are met

6 install

Installs artifact into local Maven repository (~/.m2)

7 deploy

Copies final artifact to remote repository for sharing

Each phase triggers all previous phases automatically

Parent POM

Centralizing configuration across multi-module projects

What is a Parent POM?

A Parent POM is a special POM that other projects inherit from. Child modules automatically receive all dependency versions, plugin configs, and properties defined in the parent, ensuring DRY (Don't Repeat Yourself) principles across large projects.

Key Benefits:

- Centralized dependency version management
- Shared plugin configuration
- Consistent build behavior
- Single place to update versions
- Spring Boot provides its own parent POM

```
<!-- Parent POM -->  
<parent>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-parent</artifactId>  
  <version>3.2.0</version>  
  <relativePath/>  
</parent>  
  
<!-- Child inherits all configs -->  
<!-- No need to repeat versions -->
```

Dependency Scope

Controlling when and where dependencies are available

Scope	Compile	Test	Runtime	Description
compile	✓	✓	✓	Default scope. Available everywhere — compile, test, and runtime classpaths.
provided	✓	✓	✗	Available at compile/test but NOT bundled. JEE containers (e.g., Tomcat) provide it at runtime.
runtime	✗	✓	✓	Not needed to compile but needed at runtime. JDBC drivers are a typical example.
test	✗	✓	✗	Only for test compilation and execution. JUnit, Mockito are declared test scope.
system	✓	✓	✗	Like provided but references a JAR via explicit path. Discouraged — not portable.
import	—	—	—	Only for <dependencyManagement>. Imports a BOM (Bill of Materials) from another POM.

Transitive Dependencies & Version Conflicts

How Maven resolves the dependency graph

Transitive Dependencies

When you declare $A \rightarrow B$, Maven automatically pulls in all of B's dependencies ($B \rightarrow C$, $C \rightarrow D \dots$). You get the full tree without manually declaring each transitive dep.

Conflict Resolution — Nearest Wins

If two paths lead to the same artifact with different versions, Maven picks the version closest to the project root in the dependency tree (shortest path).

Solutions:

- Explicit direct dependency declaration
- `<exclusions>` to remove unwanted transitives
- `<dependencyManagement>` to enforce a version
- BOM (Bill of Materials) import

Excluding a Transitive Dep:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>lib-a</artifactId>
  <exclusions>
    <exclusion>
      <groupId>com.bad</groupId>
      <artifactId>old-lib</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

```
<!-- Force version via mgmt: -->
<dependencyManagement>
  <dependencies>
    <!-- pin specific version -->
  </dependencies>
</dependencyManagement>
```


Maven Plugins & Execution

Plugins are how Maven does everything

Plugins bind goals to lifecycle phases. A goal is a specific task (like `compile:compile` or `surefire:test`). Each plugin has one or more goals.

Compiler Plugin

Controls Java source/target version for compilation.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <source>17</source>
    <target>17</target>
  </configuration>
</plugin>
```

Surefire Plugin

Runs unit tests. Generates test reports. Skips with `-DskipTests`.

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.1.2</version>
  <configuration>
    <includes>**/*Test.java</includes>
  </configuration>
</plugin>
```

`mvn compile` | `mvn test` | `mvn package` | `mvn verify` | `mvn deploy`

Shade Plugin — Creating an Uber JAR

Packaging all dependencies into a single executable JAR

What is an Uber JAR?

An Uber (German for 'over/super') JAR is a single fat JAR that contains your compiled code PLUS all its dependency JARs merged inside it.

This makes deployment trivially simple — ship one file, run anywhere with `java -jar`.

When to use:

- CLI tools that need to be self-contained
- Microservices deployed to containers
- Spark/Hadoop jobs submitted to clusters
- Anywhere classpath management is painful

⚠ *Watch for class name conflicts from merged JARs*

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.5.0</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
      <configuration>
        <transformers>
          <transformer
            impl="ManifestResourceTransformer">
            <mainClass>com.App</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution></executions>
</plugin>
```

Maven Wrapper (mvnw)

Ensuring consistent Maven version across all environments

The Problem

Different developers and CI servers may have different Maven versions installed, causing 'it works on my machine' build failures.

The Solution

mvnw (Maven Wrapper) is a script committed to your repo. It auto-downloads and uses the exact Maven version specified in `.mvn/wrapper/maven-wrapper.properties`.

How to Generate

Run: `mvn wrapper:wrapper`
This creates `mvnw`, `mvnw.cmd`, and `.mvn/` directory in your project root.

Usage

```
./mvnw clean package
./mvnw spring-boot:run
# Windows: mvnw.cmd clean install
```

✓ No Maven pre-installation required ✓ Version pinned in repo ✓ Works offline (cached in `~/.m2/wrapper`)

Maven & Docker Integration

dockerfile-maven-plugin, Dockerizing apps, Pushing to registries

Integration Overview

- 1. Build JAR**
mvn package produces the application JAR in target/
- 2. Write Dockerfile**
Multi-stage Dockerfile: base image → copy JAR → ENTRYPOINT
- 3. Build Image**
dockerfile-maven-plugin runs docker build as part of mvn install
- 4. Tag Image**
Plugin tags image with project version automatically
- 5. Push to Registry**
mvn deploy pushes image to Docker Hub / ECR / GCR

Dockerfile (multi-stage):

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw package -DskipTests
```

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Build & push via Maven:
mvn dockerfile:build dockerfile:push
```

Pull Maven Image (One-Time)

```
docker pull maven:3.9.10-eclipse-temurin-17
```

```
docker run --rm -it -v ${PWD}:/app -w /app maven:3.9.10-eclipse-temurin-17
```

```
mvn archetype:generate -DgroupId=com.example -DartifactId=demo-app -  
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
mkdir test123
```

```
cd test123
```

```
docker run --rm -it -v %cd%:/app -w /app maven:3.9.10-eclipse-temurin-17 mvn  
archetype:generate -DgroupId=com.example -DartifactId=demo-app -  
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
cd demo-app
```

```
docker run --rm -it -v %cd%:/app -w /app maven:3.9.10-eclipse-temurin-17 mvn clean  
install
```